Universidad Andrés Bello | Facultad de Ingeniería | Magíster en Ciencia de Datos e Inteligencia Artificial



BIENVENIDOS

Programación para la Ciencia de Datos

Magíster en Ciencia de Datos e Inteligencia Artificial

Dr. Omar Salinas Silva | omar.salinas@unab.cl

Cómo está organizada esta presentación

Siete bloques. Los cinco primeros preparan la Formativa 1; los dos últimos, la entrega sumativa del domingo.

1	**Bienvenida al módulo**	**láminas 3–12**	Qué es el módulo, cómo se evalúa y qué hacer hoy
2	**Recorrido de la semana**	**láminas 13–17**	Qué recursos hay en el aula y en qué orden usarlos
3	**Conceptos de la Fase 1**	**láminas 18–29**	Qué comprender antes de construir el mapa conceptual
4	**El ecosistema científico**	**láminas 30–40**	Python, Jupyter, Git y GitHub: qué hace cada pieza
5	**El conjunto de datos**	**láminas 41–53**	Cómo elegirlo y cómo verificar que sirve
6	**Evaluación Formativa 1**	**láminas 54–63**	El mapa conceptual: qué se pide y cómo se evalúa
7	**Sumativa 1**	**láminas 64–77**	Los doce pasos del avance de las Fases 1 y 2

Cada bloque se sostiene solo. Los bloques 3 a 7 son las presentaciones de apoyo del aula y pueden revisarse por separado.

De qué se trata este módulo

Saber programar y saber trabajar con datos no son lo mismo. Un análisis vale por su resultado, pero también por poder demostrar cómo se llegó a él, quién hizo qué, y que otra persona pueda reproducirlo. Eso es lo que van a aprender aquí.

1

Escribir código que otros puedan leer

Funciones, módulos, manejo de errores y documentación técnica.

2

Preparar datos reales

Obtención, limpieza y transformación con NumPy y Pandas.

3

Trabajar de forma reproducible

Entornos, control de versiones y repositorios compartidos.

Qué van a lograr

Cuatro resultados de aprendizaje, uno por unidad temática.

RA1	Aplicar buenas prácticas y herramientas de software científico —GitHub, Jupyter— para la gestión y documentación reproducible.
RA2	Diseñar soluciones algorítmicas eficientes con programación estructurada y recursiva en Python.
RA3	Implementar código modular y robusto aplicando los principios de la programación orientada a objetos.
RA4	Construir visualizaciones efectivas con el ecosistema científico de Python para explorar datos y comunicar hallazgos.

Todo gira en torno a un proyecto

Aprendizaje basado en proyecto: se aprende construyendo, no acumulando temas sueltos.

F1

Definición

Problema, entorno reproducible y repositorio

→

F2

Datos

Obtención, limpieza y transformación

→

F3

Algoritmos

Núcleo algorítmico, eficiencia y POO

→

F4

Comunicación

Visualización, informe y presentación

Al final tendrán algo que mostrar

Un repositorio público con historial de trabajo real, notebooks reproducibles, un informe técnico y una presentación audiovisual. No un ejercicio de clase: un proyecto que pueden poner en su portafolio profesional.

Cómo se organizan las tres semanas

Cada semana cierra con una entrega. El ritmo es exigente desde el primer día.

Semana 1

2 al 8 de noviembre

Fundamentos de Python · NumPy y Pandas

Fases 1 y 2: definición del proyecto, entorno reproducible y pipeline de datos

40%

Cierre de cada entrega · domingo a las 23:59 Sumativa 1: 8 de noviembre · Sumativa 2: 15 de noviembre · Sumativa 3: 22 de noviembre

Semana 2

9 al 15 de noviembre

Programación estructurada y recursiva · POO

Fase 3: núcleo algorítmico, eficiencia y orientación a objetos

20%

Semana 3

16 al 22 de noviembre

Visualización y comunicación de resultados

Fase 4: informe final, visualizaciones y presentación audiovisual

40%

La Semana 1 concentra dos unidades temáticas y el 40% de la nota. No hay receso: la Fase 3 parte el lunes 9, al día siguiente de esa entrega.

Cómo se evalúa

Cuatro instancias sumativas, todas sobre el mismo proyecto. Nota mínima de aprobación: 4,0.

Evaluación	Qué se entrega	Semana	Pond.
Sumativa 1 — Fases 1 y 2	Informe, repositorio y notebooks del entorno y el pipeline de datos	1	40%
Sumativa 2 — Fase 3	Núcleo algorítmico con eficiencia y POO	2	20%
Sumativa 3 — Fase 4	Proyecto final: informe, notebooks y presentación audiovisual	3	30%
Autoevaluación y coevaluación	Escala de apreciación sobre el trabajo del equipo	3	10%

Hay también evaluaciones formativas

No ponderan, pero su retroalimentación se incorpora a la entrega calificada siguiente. Es la única instancia donde equivocarse no cuesta nota.

Todo se evalúa con rúbrica

Está publicada en cada tarea, antes de la entrega. Ábranla al empezar, no al terminar: dice exactamente qué se espera en cada criterio.

El trabajo es en equipo, y eso se evalúa

Grupos de 3 o 4 integrantes, que se mantienen durante todo el curso.

1

Formen el grupo el primer día

Sin grupo no hay avance posible: la primera entrega es al cierre de la Semana 1.

2

Todos trabajan sobre el mismo repositorio

El historial de commits muestra quién hizo qué y cuándo. Es evidencia evaluada.

3

Repártanse por archivo, no por línea

Dos personas editando el mismo notebook a la misma hora garantiza conflictos.

4

Hay coevaluación

El 10% final lo determina cómo evalúan el aporte de cada integrante, incluido el propio.

El patrón que más problemas causa: una persona hace el trabajo y lo sube por todos. Produce un repositorio con un solo autor, y eso baja la nota del grupo completo por más completo que esté el contenido.

Cómo funciona la modalidad online

Trabajo autónomo con acompañamiento. El aula está ordenada como una secuencia.

Recursos por semana

Cápsulas, videos, interactivos y apuntes descargables, en un orden pensado.

Sesión sincrónica

Encuentro semanal para reforzar contenidos y resolver consultas. Queda grabada.

Foro de Consultas

Dudas de contenido y errores técnicos. Es el canal principal.

Foro de Tutoría

Problemas de acceso, plataforma y soporte.

Foros de reflexión

Uno por fase, para discutir decisiones técnicas con el grupo curso.

Anuncios

Recursos y avisos que no están dentro de los módulos. Revísenlos.

Dedicación estimada: unas 12 horas semanales entre lectura, práctica y trabajo grupal. Distribuidas en la semana, no concentradas en el último día.

Qué necesitan tener listo

Cinco cosas, todas gratuitas. Déjenlas resueltas antes de la primera entrega.

1	**Python 3.13**	Descarga desde python.org. En Windows, marquen «Add python.exe to PATH».
2	**Git**	En Windows incluye Git Bash; en macOS viene con las Command Line Tools.
3	**Cuenta de GitHub**	Con el mismo correo que configuren en Git, para que sus commits se les atribuyan.
4	**JupyterLab**	Se instala dentro del entorno virtual del proyecto, no en el Python del sistema.
5	**Un conjunto de datos**	Elegido por el grupo según los criterios del curso. Verificado antes de comprometerse.

La Guía de Git y GitHub del curso, en versión Windows y macOS, cubre la instalación paso a paso.

Integridad académica y uso de IA

Dos reglas claras, para que nadie las descubra tarde.

El trabajo debe ser propio

El plagio se sanciona con nota mínima, sin posibilidad de reposición.

El historial de Git es un registro de autoría: un repositorio cuyo primer commit contiene el proyecto terminado es, como mínimo, una señal de que el proceso no ocurrió como se declara.

El uso de IA debe ser transparente

Si usan un asistente para generar o depurar código, declárenlo según los lineamientos publicados en el curso.

Y asegúrense de entender y poder explicar cada línea que entregan: la evaluación distingue entre obtener un resultado correcto y comprender por qué lo es.

Por qué importa en este magíster

En ciencia de datos, la trazabilidad no es un requisito administrativo: es lo que hace que un resultado sea creíble. Las mismas prácticas que aquí se evalúan —documentar decisiones, versionar el trabajo, declarar las fuentes— son las que sostienen la validez de cualquier análisis profesional.

Los primeros pasos, hoy mismo

Cinco acciones que se hacen en menos de una hora y ordenan toda la semana.

1	**Recorrer el aula**	Bienvenida, información general y programa de la asignatura.
2	**Completar el compromiso**	Cuestionario de integridad académica: un intento, sin puntaje.
3	**Participar en el foro diagnóstico**	Activación de conocimientos previos. No pondera, pero orienta el curso.
4	**Formar el grupo**	3 o 4 integrantes, para todo el semestre.
5	**Abrir la rúbrica de la Sumativa 1**	Antes de empezar a trabajar, no después de entregar.

Quien parte el primer día llega con holgura a la entrega. Quien parte el jueves, no alcanza a montar el repositorio con calma.

Universidad Andrés Bello · Facultad de Ingeniería · Magíster en Ciencia de Datos e Inteligencia Artificial



SEMANA 1

Fundamentos avanzados de Python y buenas prácticas

Recorrido por los recursos del aula, en el orden en que están dispuestos

Una semana, dos fases del proyecto

Ambas se llaman «Semana 1» en el aula. No dicen lo mismo.

Fase 1

Definición y orientación de la situación

Definir la problemática y planificar el flujo de trabajo reproducible mediante la identificación del contexto, los objetivos iniciales y la organización preliminar del entorno colaborativo.

30 min de lectura · 7 recursos

Fase 2

Búsqueda y recopilación de información

Implementar un pipeline inicial de obtención, limpieza y transformación de datos en un notebook ejecutable y documentado dentro de un repositorio GitHub.

90 min de lectura · 10 recursos

La progresión es deliberada: la Fase 1 planifica el ecosistema técnico; la Fase 2 lo implementa sobre datos reales. Lo definido en la primera es lo que la segunda materializa en código.

Fase 1 · Qué aporta cada recurso

Recurso	Qué permite hacer
Introducción Fase 1	Comprender el enfoque ABP, el tipo de problemática y el rol de las decisiones metodológicas
Pensamiento algorítmico	Transformar la problemática en un flujo lógico de procesos, decisiones y etapas programables
Infografía del mapa conceptual	Saber cómo representar problema, flujo, herramientas y sus relaciones
Ecosistema científico	Reconocer las herramientas clave de Python para preparar, modelar y visualizar
Jupyter Notebooks	Entender cómo se integran análisis, código y documentación en un entorno reproducible
Git y GitHub (interactivo)	Comprender el control de versiones y cómo se asegura la trazabilidad del trabajo
Git y GitHub (audiovisual)	Ver ejemplos concretos de versionamiento y uso del repositorio como documentación

El recurso de pensamiento algorítmico es el que evita el error más frecuente: un mapa con descripciones genéricas, desconectado del enfoque técnico que exige la fase.

Fase 2 · Objetivo y resultados de aprendizaje

El foco cambia: de la planificación conceptual al trabajo técnico directo con datos.

Desarrollar la capacidad de obtener, limpiar y transformar conjuntos de datos reales con NumPy y Pandas, integrando estructuras de control y funciones para automatizar procesos, documentar decisiones técnicas y asegurar calidad, coherencia y reproducibilidad.

RA4

Construir visualizaciones de datos efectivas con librerías del ecosistema científico de Python.

ID 4.1 · Aplican técnicas de preprocesamiento y transformación con NumPy y Pandas, asegurando calidad y consistencia.

RA2

Diseñar soluciones algorítmicas eficientes con programación estructurada y recursiva en Python.

ID 2.1 · Codifican scripts o prototipos que integren datos preprocesados con estructuras de control y funciones.

Ambos indicadores se evidencian en el mismo producto: el notebook ejecutable e integrado al repositorio.

Ruta sugerida para la semana

Respetando el orden en que los recursos están dispuestos en el aula.

Día 1	Fase 1: introducción, pensamiento algorítmico e infografía del mapa.	1 h
Día 2	Fase 1: ecosistema científico, Jupyter y los dos recursos de Git y GitHub.	1 h
Día 3	Elaborar y entregar la Formativa 1. Revisar entornos reproducibles.	1,5 h
Día 4	Fase 2: introducción, programación estructurada, funciones y sus apuntes.	2 h
Día 5	Fase 2: apuntes de NumPy y Pandas, más los recursos de preprocesamiento.	2 h
Días 6 y 7	Implementar el pipeline, montar el repositorio y redactar el informe.	4 h

Estimación referencial de unas 11 horas. El programa considera 36 horas personales para las tres semanas del curso.

Universidad Andrés Bello · Facultad de Ingeniería · Magíster en Ciencia de Datos e Inteligencia Artificial



CONCEPTOS TEÓRICOS

Fase 1 · Definición y orientación de la situación

Qué hay que comprender antes de construir el mapa conceptual

Seis conceptos sostienen toda la Fase 1

Cada uno responde una pregunta distinta. Juntos explican por qué el proyecto se organiza así.

1

Contexto problematizador

¿De qué situación parte el proyecto?

2

Ciclo de vida del dato

¿Por qué etapas pasa un análisis?

3

Reproducibilidad

¿Puede otra persona obtener el mismo resultado?

4

Trazabilidad

¿Se puede reconstruir cómo se llegó aquí?

5

Documentación científica

¿Dónde queda registrado el razonamiento?

6

Mapa conceptual

¿Cómo se representa todo lo anterior?

Qué significa «definición y orientación de la situación»

La Fase 1 del aprendizaje basado en proyecto tiene una función precisa: acotar antes de resolver.

En un proyecto, definir es decidir

- Qué problema se aborda y cuál queda fuera.
- Con qué datos se va a trabajar y cuáles no están disponibles.
- Qué se considerará una respuesta satisfactoria.
- Con qué medios técnicos se va a producir esa respuesta.

Por qué se define antes de ejecutar

En ciencia de datos, el costo de un error crece con la etapa en que se descubre.

Un problema mal delimitado en la Fase 1 no se manifiesta hasta que hay código escrito, datos transformados y resultados calculados sobre una pregunta equivocada.

Rehacer entonces cuesta el trabajo de tres fases, no el de una.

La Fase 1 no produce análisis. Produce las condiciones para que el análisis sea posible, verificable y compartido.

Del contexto problematizador a la pregunta analizable

Una situación no es una pregunta, y una pregunta no siempre es analizable con datos.

1

Situación

Existe un fenómeno que preocupa o interesa.

«Hay muchos casos en la región.»

→

2

Problema

La situación se delimita: qué parte se aborda.

«No se sabe qué factores concentran los casos.»

→

3

Pregunta analizable

La pregunta se puede responder con los datos disponibles.

«¿Qué variables se asocian a la ocurrencia en este registro?»

Tres condiciones de una pregunta analizable

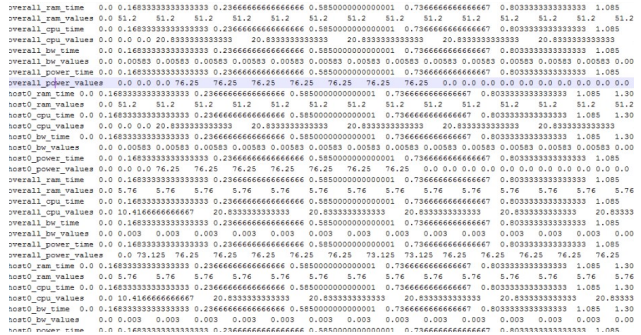
Los datos necesarios existen y son accesibles.

Los conceptos involucrados se pueden medir con esas variables.

El alcance está acotado: se sabe qué queda fuera.

El ciclo de vida de un proyecto de datos

Las etapas no son burocráticas: cada una transforma el dato y deja evidencia de esa transformación.



1. **Definición**
   Se acota el problema y se identifican los datos.
2. **Obtención**
   Se accede a las fuentes y se incorpora el dato crudo.
3. **Exploración**
   Se examina la estructura, la calidad y las anomalías.
4. **Transformación**
   Se limpia, se corrige y se derivan nuevas variables.
5. **Análisis**
   Se aplican procedimientos para responder la pregunta.
6. **Comunicación**
   Se interpretan y se transmiten los hallazgos.

Dos propiedades del ciclo

No es lineal en la práctica: se vuelve atrás cada vez que la exploración revela algo que obliga a redefinir. Y el dato crudo nunca se modifica: cada etapa produce una versión nueva, de modo que siempre se puede volver al origen y demostrar qué se transformó.

Reproducibilidad: el concepto central de la unidad

Es la propiedad que distingue un resultado verificable de una afirmación sin respaldo.

Un trabajo es reproducible cuando otra persona, con los mismos datos y el mismo código, obtiene exactamente el mismo resultado.

Reproducibilidad

Mismos datos, mismo código, mismo resultado.

Es lo que se exige en este curso.

Replicabilidad

Datos nuevos, mismo procedimiento, conclusiones equivalentes.

Pertenece al diseño de la investigación.

Tres condiciones que deben cumplirse a la vez

Los datos

Disponibles, o su origen documentado con precisión.

El código

Completo, ordenado y ejecutable de principio a fin.

El entorno

Las versiones de las librerías, declaradas explícitamente.

Trazabilidad: poder reconstruir el camino

La reproducibilidad mira el resultado. La trazabilidad mira el proceso que llevó a él.

Qué debe poder reconstruirse

- Qué cambió en el proyecto.
- Cuándo se produjo ese cambio.
- Quién lo hizo.
- Por qué se decidió hacerlo.

El control de versiones como concepto

Es la práctica de registrar la evolución de un proyecto como una secuencia de estados identificables, en lugar de una sucesión de archivos sobrescritos.

Git es una implementación de esa práctica; la práctica es anterior a la herramienta y más importante que ella.

Por qué importa en trabajo grupal

La trazabilidad hace visible la contribución individual dentro de un producto colectivo. Sin ella, el trabajo del equipo aparece como una masa indistinta y no hay forma de reconocer —ni de evaluar— el aporte de cada integrante.

Documentación científica y narrativa computacional

El notebook no es un editor de código: es un formato de comunicación científica.

La idea de fondo

Un análisis no se comunica solo con su código, ni solo con sus resultados. Se comunica con el razonamiento que los une.

El notebook materializa esa idea: el texto explica, el código ejecuta y la salida evidencia, en un mismo documento y en el mismo orden en que ocurrió el pensamiento.

Dos niveles de documentación

Descriptivo

Registra qué se hizo. Es necesario, pero insuficiente.

Justificativo

Registra por qué se decidió hacerlo así y qué alternativas se descartaron.
El nivel justificativo es el que exige el trabajo académico.

Ejemplo de la diferencia

«Se imputó la mediana» describe. | «Se imputó la mediana porque el 9% de los valores falta y la variable presenta valores extremos que desplazarían la media» justifica.

Buenas prácticas de software científico

Los contenidos de la unidad temática 1, leídos como principios y no como sintaxis.

1

Funciones

Encapsular una tarea con entradas y salidas explícitas.

Un análisis escrito como una sucesión de celdas sueltas no se puede reutilizar ni verificar por partes.

2

Módulos

Separar el código reutilizable del documento narrativo.

Lo que se usa en varias etapas vive en un archivo propio, no copiado tres veces.

3

Excepciones

Prever y gestionar lo que puede fallar.

Los datos reales traen valores anómalos: el código debe decidir qué hacer con ellos, no detenerse.

El principio común: un programa científico se escribe para ser leído, revisado y ejecutado por otros, no solo por quien lo escribió.

El mapa conceptual como instrumento

No es un resumen ilustrado: es una representación de relaciones entre conceptos.

1

Conceptos

Las ideas, en nodos. Sustantivos, no frases.

2

Conectores

Las líneas que unen. Llevan una relación explícita.

3

Proposiciones

Concepto + conector + concepto forman una afirmación.

4

Jerarquía

Lo general arriba, lo específico abajo o hacia fuera.

La prueba de un buen conector

Una flecha sin etiqueta no dice nada. «Git ——→ GitHub» es ambiguo; «Git ——sincroniza con——→ GitHub» es una proposición verificable. Si al leer una flecha en voz alta no resulta una frase con sentido, esa relación todavía no está definida.

Del concepto a la evidencia evaluada

Cada concepto teórico se manifiesta en algo concreto del proyecto.

Concepto	Evidencia en el proyecto	Indicador
Contexto problematizador	Problema y dataset declarados y acotados	ID 1.1
Ciclo de vida del dato	Secuencia de etapas representada en el mapa	ID 1.1
Reproducibilidad	Entorno, dependencias y ejecución verificable	ID 1.2
Trazabilidad	Historial de cambios con autoría identificable	ID 1.2
Documentación científica	Código, narrativa y referencias articulados	ID 1.3

Los conceptos no se evalúan preguntando su definición: se evalúan observando si el trabajo los aplica.

Compruebe su comprensión

Si puede responder estas seis preguntas, tiene la base conceptual para elaborar el mapa.

1. ¿Por qué una situación no equivale todavía a una pregunta analizable?
2. ¿Qué diferencia hay entre reproducibilidad y replicabilidad?
3. ¿Puede un trabajo ser reproducible y no trazable? ¿Y al revés?
4. ¿Qué distingue documentar «qué se hizo» de documentar «por qué se hizo»?
5. ¿Por qué el ciclo de vida del dato no es realmente lineal?
6. ¿Qué convierte una flecha de su mapa en una proposición verificable?

Estas preguntas son también un buen punto de partida para las intervenciones en el foro de reflexión de la fase.

Universidad Andrés Bello · Facultad de Ingeniería · Magíster en Ciencia de Datos e Inteligencia Artificial



EL ECOSISTEMA CIENTÍFICO

Python · Jupyter · Git · GitHub

Cuatro herramientas, una sola forma de trabajar

El problema que este ecosistema resuelve

Casi todos hemos tenido esta carpeta.

```
analisis_final.ipynb
analisis_final_v2.ipynb
analisis_final_v2_CORREGIDO.ipynb
analisis_final_ESTE_SI.ipynb
analisis_ESTE_SI_camila.ipynb
analisis_definitivo_ok.ipynb
```

Tres preguntas que esa carpeta no puede responder

1. ¿Qué cambió entre una versión y otra?
2. ¿Por qué se hizo ese cambio?
3. ¿Cuál era el estado el día en que sí funcionaba?

En un trabajo grupal el problema se multiplica: además, nadie sabe qué versión tiene el resto. El ecosistema científico existe para que el proceso quede registrado, no solo el resultado.

Cuatro piezas, cuatro funciones distintas

Ninguna reemplaza a otra. La confusión más común es tratarlas como si fueran lo mismo.

1

Python

El lenguaje

Ejecuta el análisis: transforma los datos y produce los resultados.

2

Jupyter

El cuaderno

Reúne código, resultados y explicación en un solo documento.

3

Git

El historial

Registra qué cambió, cuándo y por quién, en su computador.

4

GitHub

El punto de encuentro

Aloja el repositorio en la nube y permite trabajar en equipo.

Git ≠ GitHub

Git funciona en su computador y sin internet. GitHub es un servicio que aloja repositorios Git. Puede usar Git sin GitHub; no al revés.

Python: el lenguaje y su entorno

El código es solo una parte. Lo que hace reproducible el análisis es el entorno donde corre.

El intérprete

Ejecuta las instrucciones. En macOS y Linux se invoca como python3; en Windows, como python.

Las librerías

NumPy, pandas, Matplotlib, scikit-learn. No vienen con Python: se instalan, y cada una tiene su versión.

El entorno virtual

Una instalación aislada, propia del proyecto. Evita que las versiones de un trabajo rompan las de otro.

Un entorno por proyecto

```
python -m venv .venv          crea el entorno
source .venv/bin/activate     lo activa (macOS)
python -m pip install pandas  instala dentro de él
```

Por qué importa para el curso

Si cada integrante instala versiones distintas, el mismo notebook produce resultados distintos. El entorno declarado es la condición para que el análisis sea verificable.

Jupyter Notebook: código, resultado y explicación

Es el formato natural de la comunicación científica: el razonamiento va junto al cálculo.

Anatomía de un notebook

Celda Markdown

```
## Limpieza de valores faltantes
Se imputa la mediana porque...
```

Celda de código

```
df["imc"] = df["imc"].fillna(mediana)
```

Salida

```
228 filas procesadas · 0 nulos restantes
```

Dos cosas que conviene saber

El kernel

Es el intérprete de Python que ejecuta el notebook. Debe ser el del entorno del proyecto, no el genérico «Python 3»: ahí se origina la mayoría de los «a mí me funciona».

El orden de ejecución

Las celdas pueden ejecutarse en cualquier orden, y eso rompe la reproducibilidad. Antes de entregar: Kernel → Restart Kernel and Run All Cells.

Los contadores In [7], In [3], In [12] delatan un notebook que solo funciona en el orden en que su autor lo fue ejecutando.

Dónde se rozan Jupyter y Git

El punto que las guías genéricas omiten y que en trabajo grupal aparece la primera semana.

Un archivo .ipynb guarda, dentro del mismo JSON, las salidas de cada celda y los contadores de ejecución.

Diferencias ilegibles

Cambiar una línea genera cientos de líneas de diferencia, porque también cambian las imágenes codificadas.

Conflictos constantes

Dos personas que solo ejecutaron el notebook, sin editarlo, ya tienen versiones distintas.

Datos expuestos

Una tabla o un gráfico impreso puede dejar información sensible en un repositorio público.

Cómo se resuelve

nbstripout limpia las salidas antes de cada commit: el notebook en pantalla no cambia, solo se limpia lo que va al historial. Para la entrega, se exporta aparte el notebook ejecutado.

Git: por dónde pasa un archivo antes de quedar guardado

Tres estados. Entenderlos evita la mayoría de las confusiones posteriores.

1 Working Directory

Usted edita y guarda archivos. Git ve el cambio pero no lo ha registrado.

→ `git add`

2 Staging Area

Zona intermedia: aquí elige qué cambios entran en el próximo registro.

→ `git commit`

3 Committed

Git guardó esa versión en el historial, con autor, fecha y mensaje.

El ciclo diario

```
git status    ¿qué cambió?
git add .     preparar los cambios
git commit -m "Describe el avance"
git log --oneline    ver el historial
```

El mensaje del commit importa

«primer commit» no dice nada.
«docs: crea estructura inicial del proyecto» sí.

La prueba: si alguien lee solo su historial, ¿entiende cómo evolucionó el trabajo?

GitHub: donde el trabajo se vuelve colectivo

Git registra su historia local. GitHub la pone en común con el equipo.

1

Repositorio remoto

Una copia del proyecto en la nube, con todo el historial. Es el respaldo y el punto de referencia común.

2

Colaboradores

Cada integrante recibe acceso de escritura y trabaja sobre el mismo repositorio, con sus propios commits.

3

Ramas

Líneas de trabajo paralelas: se experimenta sin tocar la versión estable del proyecto.

4

Pull Request

Propuesta de integrar una rama, abierta a revisión. Deja registro de la discusión, no solo del código.

El historial muestra quién hizo qué y cuándo. En el proyecto del curso eso no es un detalle administrativo: es la evidencia de que el trabajo fue efectivamente grupal.

Cómo se coordinan entre sí

Tres piezas viven en su computador. Una vive en la nube. Dos comandos las conectan.

EN SU COMPUTADOR

Python + entorno virtual

ejecuta el código y aísla las versiones

Jupyter Notebook

escribe el análisis y su explicación

Git

registra cada cambio en el historial local

`git push →`

sube sus commits

`← git pull`

trae lo de los demás

EN LA NUBE

GitHub

Repositorio remoto compartido por el equipo:

- historial completo
- trabajo de los 4 integrantes
- respaldo del proyecto

El notebook produce el análisis · Git lo versiona · GitHub lo comparte · El entorno hace que corra igual en los cuatro computadores

Cuando dos personas tocan lo mismo

Un conflicto no es un error: es Git pidiendo una decisión que solo el equipo puede tomar.

Así se ve un conflicto

```
<<<<<<< HEAD
df = df.dropna(subset=["edad"])
=======
df["edad"] = df["edad"].fillna(mediana)
>>>>>>> f2-limpieza-datos
```

Arriba su versión, abajo la del compañero.

Qué hacer

- Conversar: en el ejemplo, eliminar filas o imputar la mediana no son equivalentes.
- Editar el archivo dejando la versión correcta y borrar las líneas de marcas.
- Confirmar con git add y git commit.

Prevenir

nbstripout activo en los cuatro equipos reduce drásticamente los conflictos en notebooks.

Repartir

Trabajar por archivo, no por línea: dos personas en el mismo notebook a la misma hora chocan siempre.

Practicar

Provoquen un conflicto a propósito en la semana 1, no la noche antes de entregar.

Dónde falla el acoplamiento entre las piezas

Casi ningún problema viene de una herramienta sola: viene de cómo se conectan.

Síntoma	Dónde está la falla	Corrección
«A mí me funciona y a ti no»	Kernel genérico en vez del kernel del proyecto	Seleccionar el kernel del entorno virtual
ModuleNotFoundError con un paquete instalado	Entorno virtual no activado	Activar el entorno y verificar que aparezca (.venv)
El notebook no corre en otro equipo	Rutas absolutas en el código	Usar rutas relativas desde la raíz del proyecto
El repositorio pesa cientos de MB	Se versionó la carpeta del entorno	Declararla en .gitignore antes del primer commit
Conflictos permanentes en el .ipynb	Las salidas del notebook van al historial	Activar nbstripout en cada equipo
push rechazado (non-fast-forward)	El remoto tiene commits que usted no tiene	Hacer git pull y resolver. Nunca --force

Universidad Andrés Bello · Facultad de Ingeniería · Magíster en Ciencia de Datos e Inteligencia Artificial



EL CONJUNTO DE DATOS

Cómo elegirlo y cómo verificar que sirve

La primera decisión técnica del proyecto transversal

La elección del dataset condiciona todo el proyecto

Es la única decisión de la Fase 1 que no se puede revertir sin rehacer el trabajo.

1

Determina la pregunta

Solo se puede preguntar lo que los datos permiten responder.

2

Determina el trabajo técnico

Si no hay nada que limpiar, no hay preprocesamiento que justificar.

3

Determina el interés

Un problema contingente sostiene la motivación de tres semanas.

En la Fase 2 se limpia y transforma. En la Fase 3 se construye el núcleo algorítmico. En la Fase 4 se comunica. Las tres trabajan sobre el conjunto de datos que ustedes eligen esta semana.

Dedíquenle tiempo real: buscar y validar un buen dataset puede tomar más que dibujar el mapa conceptual, y vale más.

El conjunto de datos del proyecto

Debe ser multivariado y combinar variables de distinto tipo. Cada tipo obliga a un tratamiento distinto, y ese tratamiento es lo que se evalúa.

Debe contener

Binarias 0/1, sí/no
Codificación 0-1

Enteras recuento discreto
Decidir si se tratan como número o como categoría

Flotantes magnitud continua
Valores atípicos, escalamiento e imputación

Categóricas nominales sin orden natural
One-hot encoding y agrupación de categorías raras

Categóricas ordinales con orden natural
Codificación con el orden declarado por el grupo

A esos cinco tipos, los requisitos mínimos del curso suman una variable de fecha y una de alta cardinalidad.

No se acepta

× **Titanic, Iris, Housing**

Conjuntos didácticos de circulación masiva: vienen pre-limpiados y tienen miles de soluciones publicadas.

× **Stroke**

Es el conjunto trabajado en clase. El caso docente queda excluido del proyecto.

× **Kaggle ya preprocesado**

Los preparados para una competencia no dejan preprocesamiento real que hacer. Si eligen Kaggle, que sean datos crudos.

Quedan excluidos además los datos personales identificables, el origen no verificable y la repetición del mismo conjunto entre grupos.

Un conjunto ya limpio no es una ventaja. Si no hay valores faltantes, categorías inconsistentes ni decisiones que tomar, no hay nada que justificar, y el criterio de mayor peso de la Sumativa 1 queda sin evidencia.

Ocho roles, ocho tratamientos distintos

Su dataset debe permitirles practicar todos. Esa es la exigencia real.

Rol analítico	Qué preprocesamiento exige	Ejemplo
Numérica continua	Outliers, escalamiento, imputación por media o mediana	Monto, temperatura
Numérica discreta	Decidir si se trata como número o como categoría	N.° de eventos
Binaria	Codificación 0/1 y revisión de desbalance	Aprobado / rechazado
Categórica nominal	One-hot encoder. Grupar categorías poco frecuentes	Región, sector
Categórica ordinal	LabelEncoder. Codificación que preserve el orden declarado	Riesgo: bajo, medio, alto
Fecha	Parseo y derivación de año, mes, antigüedad	Fecha de ingreso
Alta cardinalidad	Agrupación o codificación por frecuencia	Comuna, rubro
Identificador	Verificar unicidad y excluir del análisis	Folio, RUT anonimizado

La categórica ordinal es la que más se omite: exige declarar el orden explícitamente, no dejar que la librería lo invente alfabéticamente.

Requisitos mínimos del dataset

Si su conjunto de datos no los cumple, todavía no es el dataset del proyecto.

2.000 filas como mínimo	**12** columnas como mínimo	**2** numéricas continuas	**1** numérica discreta
2 categóricas nominales	**1** binaria u ordinal	**1** variable de fecha	**1** texto o alta cardinalidad

- Ninguna variable sobre 60% de valores faltantes
- Licencia abierta y fuente verificable
- Archivo bajo 100 MB (límite de GitHub)
- Datos de los últimos cinco años

Un noveno requisito, el menos evidente, va en la lámina siguiente.

El dataset debe tener datos faltantes

Suena raro. Es el requisito que más protege su nota.

Dataset «perfecto»

Sin nulos, sin duplicados, sin anomalías.

La sección de preprocesamiento se reduce a cargar el archivo. No hay decisiones que tomar, y por lo tanto no hay nada que justificar.

El criterio de mayor peso de la Sumativa 1 queda sin evidencia.

Dataset real

Con campos vacíos, formatos inconsistentes y categorías escritas de tres maneras.

Cada uno obliga a decidir: eliminar, imputar, corregir. Y cada decisión se justifica en el informe.

Eso es exactamente lo que la rúbrica evalúa.

La proporción también importa

Menos de 1%

Casi no hay decisión que tomar.

Entre 1% y 30%

Rango útil: obliga a elegir entre eliminar e imputar, y a justificarlo.

Más de 60%

La variable no es recuperable: se descarta.

Un problema contingente, no un ejercicio

Los datos deben permitir decir algo sobre una situación que está ocurriendo.

Contingente = tiene consecuencias hoy para alguien. Si el análisis no le interesaría a ningún tomador de decisiones, probablemente el problema está mal elegido.

1. **Salud pública y acceso**

   Variación territorial en atención de urgencia o listas de espera

2. **Transparencia y compra pública**

   Patrones en licitaciones con un solo oferente

3. **Trayectorias educativas**

   Factores que acompañan la deserción o el cambio de establecimiento

4. **Fiscalización ambiental**

   Unidades con procesos sancionatorios recurrentes

5. **Transición energética**

   Distribución de la generación renovable y sus condicionantes

6. **Movilidad y logística urbana**

   Diferencias de desempeño entre nodos logísticos

Son líneas de búsqueda, no temas cerrados. Pueden proponer otra, siempre que tenga datos abiertos disponibles.

Dónde buscar: fuentes chilenas

Datos abiertos, licencia clara y contingencia local garantizada.

Fuente	Qué contiene	Sitio
Portal de Datos Abiertos	Repositorio central del Estado, 255+ organismos	datos.gob.cl
DEIS · MINSAL	Mortalidad, vacunación, egresos, urgencias	deis.minsal.cl
ChileCompra	Compras públicas: montos, rubros, organismos	datosabiertos.chilecompra.cl
Datos Abiertos MINEDUC	Estudiantes, docentes, establecimientos	datosabiertos.mineduc.cl
SNIFA · SMA	Fiscalizaciones, sanciones, emisiones	snifa.sma.gob.cl
Energía Abierta · CNE	Electricidad, renovables, eficiencia	energiaabierta.cl
Observatorio Logístico	Transporte carretero, marítimo y aéreo	observatoriologistico.cl

Lista curada y mantenida de fuentes chilenas: github.com/imfd/awesome-data-chile

Cuatro conjuntos de datos concretos

Enlace, unidad de observación y qué roles de variable esperan encontrar.

Inside Airbnb — Santiago

`insideairbnb.com/get-the-data`

Unidad: Un aviso de alojamiento publicado en Santiago

Precio y latitud (continuas) · dormitorios y noches mínimas (discretas) · superanfitrión (binaria) · tipo de habitación (nominal) · fecha de última reseña · barrio (alta cardinalidad) · título del aviso (texto)

El archivo detallado listings.csv.gz trae unas 75 columnas y abundantes nulos en las puntuaciones de reseñas. Licencia CC0.

ChileCompra — órdenes de compra

`datosabiertos.chilecompra.cl`

Unidad: Una orden de compra del Mercado Público

Monto (continua) · cantidad de ítems (discreta) · estado (nominal) · fecha de emisión · organismo y proveedor (alta cardinalidad) · glosa del producto (texto) · identificador de orden

Archivos mensuales. Conviene acotar a un año y un sector para no superar los 100 MB.

MINEDUC — rendimiento por estudiante

`datosabiertos.mineduc.cl`

Unidad: Un estudiante en un año escolar

Promedio de notas y asistencia (continuas) · edad (discreta) · situación final (binaria) · dependencia y género (nominales) · nivel del curso (ordinal) · establecimiento y comuna (alta cardinalidad)

Las bases nacionales superan los 3 millones de filas: filtren por región o por nivel. Separador « ; » y codificación latin-1.

SNIFA — procesos sancionatorios

`snifa.sma.gob.cl/DatosAbiertos`

Unidad: Un proceso sancionatorio ambiental

Multa en UTA (continua) · número de cargos (discreta) · estado final (nominal) · categoría de la infracción (ordinal) · fechas de inicio y término · unidad fiscalizable y región

Serie 2013 en adelante. Muchos procesos abiertos dejan la fecha de término vacía: material ideal para justificar imputación.

Las características descritas corresponden a la estructura habitual de cada fuente: confirmen la versión que descarguen con el validador.

Lo que van a encontrar al descargar

Cuatro fricciones habituales en datos públicos chilenos, y cómo se resuelven.

Síntoma	Causa	Solución
Todo el archivo cae en una sola columna	El separador es « ; » y no coma	pd.read_csv(ruta, sep=";")
Las tildes salen como Ã± o Ã©	Codificación latin-1, no UTF-8	pd.read_csv(ruta, encoding="latin-1")
Los decimales se leen como texto	Coma decimal en vez de punto	decimal="," en read_csv
El archivo supera los 100 MB	Base nacional completa	Filtrar por región o año y versionar el subconjunto

Documenten el subconjunto

Si filtran la base original para que quepa en el repositorio, ese filtro es una decisión técnica y debe quedar registrada: qué criterio usaron, cuántas filas quedaron y por qué ese recorte no sesga la pregunta. Es evidencia del criterio de preprocesamiento, no un detalle administrativo.

Fuentes internacionales y cómo evaluarlas

Sirven, pero exigen más cuidado con la licencia y la vigencia.

Repositorios

Kaggle Datasets
UCI Machine Learning Repository
Zenodo
Data.gov · Eurostat
Banco Mundial · Our World in Data
Portales de datos abiertos de Argentina, México, Colombia, Brasil y Perú

Cuatro preguntas antes de descargar

- ¿Quién publicó estos datos y cuándo?
- ¿Qué licencia tienen y qué permite?
- ¿Hay un diccionario de variables?
- ¿Los datos son originales o ya vienen procesados?

La trampa de Kaggle

Muchos datasets populares de Kaggle ya vienen limpios, porque alguien los preparó para una competencia. Eso los hace cómodos y, para este curso, inservibles: no queda preprocesamiento que hacer ni decisiones que justificar. Si eligen Kaggle, prefieran conjuntos publicados como datos crudos y verifiquen la proporción de faltantes antes de comprometerse.

No decidan a ojo: validen

El curso entrega un script que verifica los requisitos y produce el informe de la entrega.

```
python validar_dataset.py data/raw/mi_dataset.csv
python validar_dataset.py datos.csv --sep ";" --encoding latin-1
python validar_dataset.py datos.csv --informe docs/validacion_dataset.md
```

1. **Perfil de variables**

   El rol analítico de cada columna, sus valores únicos y su porcentaje de nulos.

2. **Composición por tipo**

   Cuántas variables hay de cada rol.

3. **Veredicto**

   Requisito por requisito: OK, FALTA o AVISO.

4. **Informe en Markdown**

   Listo para adjuntar al entregable de la Fase 1.

Adjunten la salida del validador en el entregable. La elección del dataset deja de ser una declaración y pasa a ser una verificación.

Tener el dataset no es tener la pregunta

Una vez validado el conjunto de datos, falta el paso que define el proyecto.

1 Dataset validado

Cumple los requisitos técnicos.

→

2 Exploración inicial

Qué variables hay y qué relaciones sugieren.

→

3 Pregunta analizable

Se puede responder con estas variables y este alcance.

Tres condiciones de una pregunta analizable

Los datos necesarios existen en el conjunto elegido.

Los conceptos de la pregunta se pueden medir con esas variables.

El alcance está acotado: se sabe qué queda fuera.

Ambas cosas —el dataset y la pregunta— se declaran en el entregable de la Formativa 1.

Universidad Andrés Bello · Facultad de Ingeniería · Magíster en Ciencia de Datos e Inteligencia Artificial



FASE 1

Definición y orientación de la situación

Evaluación Formativa 1 — Mapa conceptual

Exploración guiada de herramientas colaborativas en ciencia de datos

MCDI500 · Programación para la Ciencia de Datos | Semana 1 | Trabajo grupal (3–4) | Formativa · 0% · 25 puntos

Propósito de la Fase 1

Planificar y representar conceptualmente el flujo de trabajo reproducible inicial del proyecto.

1. **Identificar** — Qué herramientas de software científico va a usar el grupo y para qué.
2. **Anticipar** — Cómo se configurará el entorno reproducible que se implementa después.
3. **Organizar** — La relación entre problema, datos, herramientas y procesos.

Resultado de aprendizaje

Aplicar buenas prácticas y herramientas de software científico para la gestión y documentación reproducible de proyectos en ciencia de datos.

ID 1.1 ID 1.2 ID 1.3

En esta fase no se programa, no se configura el control de versiones y no se documenta técnicamente el proyecto.

Todo eso ocurre en la evaluación sumativa. Aquí se planifica.

Qué indicadores se están midiendo

Los tres indicadores del RA1 atraviesan toda la actividad.

ID 1.1

Herramientas de software científico

Aplican herramientas científicas en problemas de análisis de datos en contexto profesional.

ID 1.2

Control de versiones y notebooks

Configuran Git/GitHub y la gestión con Jupyter, asegurando colaboración y reproducibilidad.

ID 1.3

Documentación colaborativa

Documentan avances integrando código, texto y referencias técnicas según buenas prácticas.

En la Fase 1 estos indicadores se evidencian de forma proyectada: el mapa muestra cómo se van a cumplir, no que ya se cumplieron.

Qué deben hacer

Trabajo grupal (3–4 integrantes) · asincrónico · dedicación estimada 90 minutos · 1 intento

1

Revisar

El material de la semana: guías técnicas, recursos disciplinares y la rúbrica.

2

Acordar

La problemática asignada y el conjunto de datos con que trabajará el grupo.

3

Elaborar

El mapa conceptual del flujo reproducible en la herramienta que elijan.

4

Verificar

Los aspectos formales y subir el archivo al buzón de tareas.

Herramientas sugeridas para el mapa

Lucidchart · XMind · MindMeister · Canva · Miro · PowerPoint u otra similar

Recomendación: elaboren primero el mapa y después redacten. Escribir suele revelar vacíos que conviene corregir antes de entregar.

Qué debe contener el mapa conceptual

Seis elementos exigidos por las instrucciones. Ninguno es opcional.

1. Representación visual del flujo de trabajo reproducible del problema y dataset asignado
2. Integración de las herramientas: GitHub, Jupyter Notebooks y control de versiones
3. Estructura conceptual del entorno técnico: repositorio, carpetas, archivos y commits proyectados
4. Vinculación entre componentes técnicos y documentación científica
5. Leyenda o glosario visual que permita interpretar el flujo
6. Evidencia de reflexión técnica grupal y coherencia con la Fase 1

Los elementos 3 y 5 son los que más se omiten: los commits proyectados y la leyenda del mapa.

Ejemplo aplicado: del enunciado al mapa

Ejemplo referencial. Cada grupo lo adapta a su problemática y a su conjunto de datos.

Estructura del repositorio proyectada

```
proyecto-grupoX-mcdi500/
├─ data/
│   ├─ raw/          datos originales
│   └─ processed/    datos transformados
├─ notebooks/        análisis documentado
├─ src/              funciones reutilizables
├─ docs/             documentación
├─ requirements.txt  dependencias
├─ .gitignore        qué no se versiona
└─ README.md         qué es y cómo se ejecuta
```

Commits proyectados

```
docs: crea estructura inicial del proyecto
docs: documenta criterios de versionamiento
data: incorpora dataset base en raw
```

Cómo se lee esto en el mapa

Carpetas → etapas

Cada carpeta corresponde a un momento del flujo, no a un capricho de orden.

Commits → trazabilidad

Los ejemplos muestran que el grupo entiende qué se registra y con qué nivel de detalle.

README → comunicación

Es el punto donde el proyecto se explica a un tercero.

.gitignore → criterio

Decidir qué NO se versiona también es una decisión técnica.

Qué deben entregar

Un solo archivo, subido por una sola persona del grupo.

Aspecto	Requisito
Formato del archivo	PDF, JPG o PNG
Extensión máxima	Dos páginas tamaño carta, horizontal o vertical
Portada	Nombres completos de todos los integrantes, como aparecerán en el acta
Tipografía	Arial o Calibri 11 pts, uso moderado de color
Referencias	Al menos una en formato APA 7
Nombre del archivo	mcdi500_s1_grupox — sin tildes, ñ ni caracteres especiales, máx. 20 caracteres
Vía de entrega	Entrega de actividad → botón «Examinar mi equipo»

El plazo importa aunque la actividad no pondere: entregar tarde significa recibir la retroalimentación cuando ya no alcanza a corregirse antes de la sumativa.

Cómo se evalúa: 5 criterios de 5 puntos

25 puntos en total. Evaluación formativa: no pondera, pero la retroalimentación se aplica a la sumativa.

1	Integración técnica del flujo reproducible y ecosistema científico	/5
2	Estructura conceptual del entorno técnico de trabajo	/5
3	Vinculación entre componentes técnicos y documentación científica reproducible	/5
4	Calidad visual, jerarquía conceptual y uso de conectores	/5
5	Reflexión técnica grupal y coherencia con la Fase 1 del ABP	/5

Niveles

Excelente	5
Bueno	3
Suficiente	2
Deficiente	1
Insuficiente	0

Del nivel Excelente al Bueno se pierden 2 puntos de golpe: la diferencia entre «completo y técnicamente preciso» y «adecuado con aspectos por profundizar».

Nivel de profundidad esperado

La misma idea, en dos niveles distintos de elaboración.

Insuficiente: solo menciona

«Usaremos GitHub para el proyecto.»

«Haremos commits.»

«El notebook tendrá texto y código.»

Nivel Excelente: integra

«GitHub aloja el repositorio remoto y permite que los cuatro integrantes trabajen sobre la misma base, con historial verificable desde la etapa de definición.»

«docs: crea estructura inicial del proyecto — usamos prefijos docs, data, feat y fix para que el historial sea legible.»

«Las celdas narrativas registran por qué se tomó cada decisión; el README explica cómo ejecutar el proyecto desde cero.»

La diferencia no es la extensión: es que la segunda versión dice para qué sirve y a qué etapa corresponde.

Errores frecuentes y cómo evitarlos

Los que más cuestan puntos, y la corrección concreta de cada uno.

Confundir Git con GitHub

Declarar la función de cada uno: control local frente a repositorio remoto y colaboración.

Mapa sin leyenda

Todo símbolo del mapa debe estar en la leyenda, y todo lo de la leyenda debe aparecer en el mapa.

Listar herramientas sin vincularlas

Asociar cada herramienta a la etapa del flujo que sostiene.

Commits genéricos

Reemplazar «primer commit» por mensajes que describan el cambio.

Mapa desconectado del problema

Partir de la problemática y el dataset: son lo que evalúa el criterio 5.

Omitir los aspectos formales

Verificar nombre de archivo, dos páginas, portada con integrantes y referencia APA 7.

Universidad Andrés Bello · Facultad de Ingeniería · Magíster en Ciencia de Datos e Inteligencia Artificial



SUMATIVA 1

Procedimientos prácticos, paso a paso

Avance del proyecto — Fases 1 y 2

MCDI500 · Programación para la Ciencia de Datos | Trabajo grupal (3–4) | 99 puntos

Qué se entrega

Cuatro productos. Los cuatro se evalúan, y ninguno reemplaza a otro.

1

Informe técnico

`PDF`

Problemática, objetivos, metodología, decisiones justificadas y bibliografía en APA 7.

2

Repositorio GitHub

`enlace`

Estructura por fases, README, .gitignore, requirements y commits de cada integrante.

3

Notebook F1

`.ipynb`

Definición del problema, entorno reproducible y estructura del proyecto.

4

Notebook F2

`.ipynb`

Pipeline de obtención, limpieza, transformación y validación.

La rúbrica evalúa los notebooks de F1 y F2 como criterios separados, de 9 puntos cada uno. Si entregan un solo archivo, debe tener las dos partes claramente rotuladas para que cada criterio pueda evaluarse por su cuenta.

Dónde se juegan los 99 puntos

Dos tercios están en el notebook y el repositorio. El resto, en el informe.

66 puntos

En el notebook

Herramientas científicas · reproducibilidad · codificación funcional · preprocesamiento · validación · notebooks F1 y F2 · vinculación del mapa

21 puntos

En el informe

Portada e índice · introducción y contextualización · problemática y objetivos · aspectos formales y APA 7

12 puntos

En el repositorio

Control de versiones · estructura, README y commits identificables por integrante

Un notebook impecable con un informe descuidado deja veintiún puntos sin cubrir.

La ruta completa: doce pasos

Los pasos 1 a 4 ya se cubrieron en los bloques anteriores. Aquí se detalla del 5 en adelante.

MONTAJE — Fase 1

1. Entorno virtual y kernel
2. Repositorio y estructura
3. Descarga y verificación del dataset
4. Diccionario de variables
5. README y requirements

PIPELINE — Fase 2

6. Exploración y medición
7. Limpieza e imputación
8. Transformación
9. Escalamiento
10. Validación y pruebas
11. Persistencia y bitácora

ENTREGA

12. Informe, commits y verificación final

El montaje del entorno y la elección del dataset pueden avanzar en paralelo. Del paso 6 en adelante, no: cada etapa consume el resultado de la anterior.

Paso 5 · README y requirements

Los dos archivos que hacen reconstruible el proyecto. Ambos son evidencia evaluada.

Qué debe decir el README

- Problemática, pregunta y objetivos
- Integrantes del grupo
- Fuente del dataset y cómo descargarlo
- Estructura del repositorio
- Requisitos y comandos de ejecución
- Criterios de reproducibilidad
- Convención de commits

requirements.txt

```
python -m pip freeze > requirements.txt
```

Regenérenlo cada vez que instalen algo nuevo, no solo antes de entregar.

Es lo que permite que otra persona reconstruya el entorno exacto:

python -m pip install -r requirements.txt

Truco que ahorra trabajo: generen el README desde el notebook, a partir de la configuración del proyecto. Así la documentación no puede desactualizarse respecto del código, y actualizar un dato lo actualiza en todas partes.

Paso 6 · Explorar antes de tocar nada

Medir primero. Todas las decisiones posteriores se justifican con estas cifras.

1. **Dimensiones y tipos**

   Cuántas filas y columnas, y de qué tipo es cada una.

2. **Valores faltantes**

   Cuántos, en qué variables y en qué proporción.

3. **Categorías observadas**

   Detecta la misma categoría escrita de dos maneras.

4. **Valores atípicos**

   Con el rango intercuartílico. Es lo que justifica imputar con mediana.

La regla de oro del preprocesamiento

No apliquen una técnica porque es la habitual: apliquen la que las cifras justifican. Si van a imputar con la mediana «porque hay valores extremos», la tabla de atípicos tiene que mostrarlos. Afirmar sin evidencia es la incoherencia que más penaliza la rúbrica.

Paso 7 · Limpieza: comparar antes de imputar

No hay un método universalmente mejor. Se comparan, se leen las cifras y se justifica.

Método	Cuándo conviene	Qué distorsiona
Eliminar filas	Faltantes escasos y aparentemente aleatorios	Pierde muestra; sesga si no son aleatorios
Media	Distribución aproximadamente simétrica	La arrastran los extremos; reduce la dispersión
Mediana	Hay valores extremos o asimetría	Reduce la dispersión; ignora las demás variables
Mediana por grupo	El valor depende de otra variable observada	Reduce la dispersión dentro de cada grupo

Qué mirar en la comparación

El cambio porcentual en la desviación estándar. Toda imputación reduce la variabilidad: el mejor método es el que menos la deforma.

Dos cosas que no pueden faltar

Una bandera que registre qué filas fueron imputadas, y el reporte del efecto sobre la dispersión para el informe.

Paso 8 · Transformación por tipo de variable

Cada rol analítico exige un tratamiento distinto. Aplicar el mismo a todo es un error de fondo.

Nominal

Sin orden natural

One-hot encoding: una columna binaria por categoría

Ejemplo: gender, work_type

Ordinal

Con orden natural

Codificación con el orden declarado explícitamente por el grupo

Ejemplo: nivel de riesgo

Derivadas

Variables nuevas

Discretizar continuas o crear banderas con umbrales del dominio

Ejemplo: grupo etario

El error que más se repite

Dejar que la librería ordene una variable ordinal por orden alfabético. «exfumador < fumador < nunca» no tiene ningún sentido en el dominio, y produce un análisis entrenado sobre una escala inventada. El orden lo declara el equipo, en un diccionario explícito.

Paso 9 · Escalamiento: comparar los tres

Igual que con la imputación: no hay uno mejor, hay uno que corresponde a sus datos.

Escalador	Qué garantiza	Cuándo conviene	Riesgo
StandardScaler	Media 0, desviación 1	Distribución aproximadamente simétrica	No acota el rango
MinMaxScaler	Rango [0, 1]	Se requiere un rango acotado	Un extremo comprime todo el resto
RobustScaler	Mediana 0, RIC 1	Hay valores extremos que no se eliminan	Las cifras no quedan «redondas»

Coherencia entre decisiones

La misma evidencia debería sostener las dos elecciones. Si reconocieron valores extremos para imputar con la mediana, tampoco tiene sentido centrar con la media al escalar.

Si eligen mediana para imputar y StandardScaler para escalar, tienen que explicar por qué el mismo hecho lleva a criterios distintos. Eso es lo que separa un pipeline pensado de una sucesión de pasos.

Paso 10 · Validar es demostrar, no afirmar

Criterio 11 de la rúbrica: 6 puntos que casi nadie cubre completos.

Validar el dataset

`assert`	No quedan valores nulos
`assert`	Todas las columnas de análisis son numéricas
`assert`	Cada grupo one-hot suma 1 por fila
`assert`	No hay filas duplicadas
`assert`	Se conservó el número de filas esperado

Probar las funciones

Caso normal	Funciona con la entrada esperada
Caso límite	Columna sin nulos, una sola categoría, conjunto vacío
Excepción	Lanza el error correcto ante una entrada inválida

Verificar la propiedad que corresponde

Cada escalador garantiza algo distinto: StandardScaler deja media 0 y desviación 1; MinMaxScaler, valores entre 0 y 1; RobustScaler, mediana 0 y rango intercuartílico 1. Comprobar la propiedad correcta es lo que convierte esto en validación y no en trámite.

Paso 11 · Guardar, verificar y documentar

El pipeline no termina con el DataFrame limpio: termina cuando queda guardado y trazable.

1 Guardar

El resultado va a data/processed/, nunca sobre data/raw/.

2 Verificar

Releer el archivo y comprobar forma y columnas. Guardar sin comprobar se descubre tarde.

3 Registrar

Exportar la bitácora de decisiones y los metadatos del proyecto.

La bitácora: el atajo para escribir el informe

Una función que anota cada decisión mientras ocurre, con sus cifras. Al escribir el informe ya no hay que reconstruir de memoria cuántos nulos imputaron ni con qué valor.

```
[F2 · limpieza] 'bmi': 201 nulos (3.9%) imputados con la mediana = 28.10;
                desviación estándar -1.98%
```

Paso 12 · El informe técnico

21 puntos que no dependen del código. Se escribe con lo que el notebook ya produjo.

Sección del informe	Qué debe contener	Criterio
Portada e índice	Formato institucional, datos completos, índice actualizado	1 · 3 pts
Introducción y contexto	Situación que origina la necesidad y relevancia del problema	2 · 6 pts
Problemática y objetivos	Pregunta técnica, objetivo general y específicos, alcance y supuestos	3 · 6 pts
Metodología y decisiones	Cada decisión del pipeline justificada con las cifras de la bitácora	7 · 6 pts
Vinculación con el mapa	Qué del mapa se implementó y qué queda proyectado	14 · 6 pts
Bibliografía	Mínimo 5 fuentes en APA 7, incluida la del conjunto de datos	15 · 6 pts

El informe y el notebook no pueden contradecirse: misma problemática, mismos objetivos, mismas cifras.

Los commits también se evalúan

12 puntos entre control de versiones y repositorio. Se construyen durante las tres semanas, no al final.

Qué mira el evaluador

- Que haya commits de cada integrante
- Que los mensajes describan el cambio
- Que estén repartidos en el tiempo
- Que exista uso de ramas cuando corresponde

Convención del curso

```
docs: crea estructura inicial del proyecto
data: incorpora dataset base en raw
feat: implementa pipeline de limpieza
fix: corrige lectura de valores N/A
test: agrega pruebas de las funciones
```

El patrón que cuesta puntos

Una persona recibe los archivos por mensajería y los sube. El repositorio queda con un solo autor y un puñado de commits gigantes la noche anterior. Por completo que esté el contenido, ese historial no alcanza el nivel superior: la solución es que los cuatro clonen el repositorio en la primera semana.

Universidad Andrés Bello · Facultad de Ingeniería · Magíster en Ciencia de Datos e Inteligencia Artificial

Antes de entregar



- El notebook corre completo tras Restart & Run All
- Las dos fases están claramente delimitadas
- Cada decisión se justifica con cifras del propio análisis
- data/raw sin modificar · data/processed con el resultado
- Commits de los cuatro integrantes, repartidos en el tiempo
- El informe dice lo mismo que el notebook
- El enlace al repositorio funciona desde otra sesión

Dudas de contenido: Foro de Consultas | Plataforma: Foro de Tutoría